

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-adeea357345537685.jsonl`
- SHA-256: `2758435fcd6db4102f2d89c188268c173b21726a71b3d1f127139a9c9a90e521`
- Source modified: `2026-06-12T10:47:25+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_c5f5bca5-d2a`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T10:43:40.288Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:

- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINES.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

A reviewer claims the following defect. ADVERSARIALLY VERIFY it by reading the actual code/files ? try to REFUTE it. Default to refute if the scenario cannot actually occur or the behavior is acceptable/intended for this tool's scope.

CLAIM [medium] Duration parity can compare source CONTAINER duration against proxy STREAM duration ? spurious failure deletes a frame-perfect proxy (C:\Users\avidu\Projects\sports-data-collector\src\core\proxy.py:130-132 and 213-219)
probe_video prefers stream.duration and falls back to format.duration. Containers like MKV/TS (OBS captures, broadcast recordings) commonly report NO per-stream duration, so the SOURCE gets the container duration = max(video, audio). The proxy is always MP4 and reports a video-stream duration. If the source's audio outlasts the video by more than the 0.2 s tolerance (common in screen/stream captures where audio stops last), verify_parity reports 'duration changed' even though the proxy is frame-identical: the good proxy is deleted after a long 4K encode and the flow hard-blocks with a misleading error and no override flag. The docstring only anticipates 'a few hundredths' of priming skew, not multi-second audio tails.

Proposed fix: Record which duration was used (stream vs container) in ProbeInfo and compare like-for-like; or when the source lacks a stream duration, derive it as nb_frames/avg_fps for the comparison; or compare proxy container duration when source only has container duration.

2. user (2026-06-12T10:43:45.581Z)

```
1      """Annotation-proxy generation for vendor handoff (ADR-002).
2
3      Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4      *frame indices* (`seconds = frame / fps` ? see `src/parsers/cvat/rally_cvat.py`).
5      A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6      vendor onto a ~30-frame sampling grid (~0.5 s) and broke the ±0.3 s boundary bar.
```

```

7 The fix is to hand the vendor a light **annotation proxy** of the *same timeline*:
8 downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9 so frame N on the proxy IS frame N on the original and returned labels apply to
10 the original unchanged.
11
12 The ADR-002 proxy contract enforced here:
13 - video timestamps pass through untouched (``-fps_mode passthrough``, never
14   ``-r``): same frame count, same fps, no trim, no leading padding;
15 - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16   failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17   golden labels;
18 - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19   survives the handoff (the indexer keys videos by file MD5; downstream
20   processing must keep running on the ORIGINAL file).
21 ""
22 import csv
23 import hashlib
24 import os
25 import shutil
26 import subprocess
27 from dataclasses import dataclass
28 from typing import Any, Dict, List, Optional
29
30
31 class ProxyError(RuntimeError):
32     """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35 #: Columns of the provenance manifest (one row appended per generated proxy).
36 MANIFEST_FIELDS = [
37     "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38     "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39     "encoder_settings", "created_at",
40 ]
41
42 # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43 # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44 # back-stepping; CRF 19 keeps the ball/shuttle legible.
45 DEFAULT_HEIGHT = 1080
46 DEFAULT_CRF = 19
47 DEFAULT_GOP = 15
48 DEFAULT_PRESET = "medium"
49
50
51 @dataclass
52 class ProbeInfo:
53     """The frame?time identity of one video stream, as ffprobe reports it."""
54     width: Optional[int] = None
55     height: Optional[int] = None
56     avg_fps: Optional[float] = None # avg_frame_rate (frames / duration)
57     r_fps: Optional[float] = None # r_frame_rate (the container's base rate)
58     nb_frames: Optional[int] = None # demuxed packet count == frame count
59     duration: Optional[float] = None # video stream duration, else container
60
61     @property
62     def resolution(self) -> str:
63         if self.width and self.height:
64             return f"{self.width}x{self.height}"
65         return "unknown"
66
67
68 def parse_rate(value: Optional[str]) -> Optional[float]:
69     """Parse an ffprobe rational rate ('60000/1001', '30/1') to float fps."""
70     if not value:
71         return None

```

```

72     try:
73         if "/" in value:
74             num, den = value.split("/", 1)
75             num_f, den_f = float(num), float(den)
76             if den_f == 0:
77                 return None
78             return num_f / den_f
79         return float(value)
80     except ValueError:
81         return None
82
83
84 def probe_video(path: str) -> ProbeInfo:
85     """Probe the frame?time identity of `path` via ffprobe (loud on failure).
86
87     Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
88     yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
89     probe, a failure here RAISES: parity verification is the whole point of the
90     proxy flow, so a missing ffprobe must stop it, not degrade it.
91     """
92     if not shutil.which("ffprobe"):
93         raise ProxyError("ffprobe not found on PATH (install FFmpeg)")
94     if not os.path.exists(path):
95         raise ProxyError(f"video not found: {path}")
96     import json as _json
97     try:
98         out = subprocess.run(
99             ["ffprobe", "-v", "error", "-select_streams", "v:0",
100              "-count_packets",
101              "-show_entries",
102              "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
103              ":format=duration",
104              "-of", "json", path],
105             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
106             ).stdout
107         doc = _json.loads(out)
108     except (subprocess.SubprocessError, OSError, ValueError) as e:
109         raise ProxyError(f"ffprobe failed on {path}: {e}")
110
111     streams = doc.get("streams") or []
112     if not streams:
113         raise ProxyError(f"no video stream found in {path}")
114     s = streams[0]
115
116     def _f(value) -> Optional[float]:
117         try:
118             return float(value)
119         except (TypeError, ValueError):
120             return None
121
122     def _i(value) -> Optional[int]:
123         try:
124             return int(value)
125         except (TypeError, ValueError):
126             return None
127
128     # Prefer the video stream's own duration; fall back to the container's
129     # (audio encoder priming can stretch the container a few hundredths).
130     duration = _f(s.get("duration"))
131     if duration is None:
132         duration = _f((doc.get("format") or {}).get("duration"))
133
134     return ProbeInfo(
135         width=_i(s.get("width")),
136         height=_i(s.get("height")),

```

```

137         avg_fps=parse_rate(s.get("avg_frame_rate")),
138         r_fps=parse_rate(s.get("r_frame_rate")),
139         nb_frames=_i(s.get("nb_read_packets")),
140         duration=duration,
141     )
142
143
144 def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:
145     """True when the stream looks variable-frame-rate (or we cannot tell).
146
147     For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
148     `frame / fps` time recompute is approximate ? conservative callers should
149     require an explicit opt-in before proxying such a source.
150     """
151     if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152         return True
153     return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154
155
156 def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157                        crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158                        preset: str = DEFAULT_PRESET) -> List[str]:
159     """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160
161     Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162     trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163     1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164     never upscales. Audio is kept (contact sound is the annotator's best
165     boundary cue).
166     """
167     return [
168         "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169         "-i", src,
170         "-map", "0:v:0", "-map", "0:a?",
171         "-vf", f"scale=-2:min({height}\\,ih)",
172         "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173         "-g", str(gop), "-pix_fmt", "yuv420p",
174         "-fps_mode", "passthrough",
175         "-c:a", "aac", "-b:a", "160k",
176         "-movflags", "+faststart",
177         dst,
178     ]
179
180
181 def run_ffmpeg(cmd: List[str]) -> None:
182     """Run an ffmpeg command, streaming its progress to the console."""
183     if not shutil.which("ffmpeg"):
184         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185     try:
186         # No capture: -stats progress and any warnings go straight to the user.
187         subprocess.run(cmd, check=True)
188     except (subprocess.SubprocessError, OSError) as e:
189         code = getattr(e, "returncode", "?")
190         raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")
191
192
193 def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:
194     """Check the proxy carries the source's exact frame?time identity.
195
196     Returns a list of human-readable failures (empty == parity holds). Frame
197     count must match EXACTLY; fps within 0.1%; duration within a small slack
198     (AAC priming can pad the container by a few hundredths of a second) that
199     still catches any real trim.
200     """
201     errors = []

```

```

202
203         if src.nb_frames is None or proxy.nb_frames is None:
204             errors.append("frame count unavailable on source and/or proxy ? cannot prove
parity")
205         elif src.nb_frames != proxy.nb_frames:
206             errors.append(f"frame count changed: source {src.nb_frames} -> proxy
{proxy.nb_frames}")
207
208         if src.avg_fps is None or proxy.avg_fps is None:
209             errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
210         elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
211             errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")
212
213         if src.duration is None or proxy.duration is None:
214             errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
215         else:
216             tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
217             if abs(proxy.duration - src.duration) > tol:
218                 errors.append(f"duration changed: source {src.duration:.3f}s -> "
f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")
219
220
221         return errors
222
223
224     def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
225         """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
226         digest = hashlib.md5()
227         with open(path, "rb") as f:
228             for chunk in iter(lambda: f.read(chunk_size), b""):
229                 digest.update(chunk)
230         return digest.hexdigest()
231
232
233     def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
234         """Append one provenance row, writing the header only on first creation."""
235         parent = os.path.dirname(os.path.abspath(manifest_path))
236         os.makedirs(parent, exist_ok=True)
237         is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
238         with open(manifest_path, "a", newline="", encoding="utf-8") as f:
239             writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
240             if is_new:
241                 writer.writeheader()
242             writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})
243

```

3. user (2026-06-12T10:43:46.916Z)

```

fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002)
4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-
proxy) (#21)
a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)
docs/INGESTION_PIPELINE.md | 29 ++++
src/cli/curate.py | 147 +++++
src/core/proxy.py | 242 +++++
tests/test_proxy.py | 309 +++++
4 files changed, 724 insertions(+), 3 deletions(-)

```

4. user (2026-06-12T10:44:18.316Z)

```

1     import os
2     import sys
3     import time
4     import json

```

```

5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27
28            # Resolve DB path
29            db_path = self.config.get("database_path", "data/sports_metadata.db")
30            self.db = SportsDatabase(db_path)
31            self.validator = DatasetValidator(config_path)
32
33        def _load_config(self) -> Dict[str, Any]:
34            if not os.path.exists(self.config_path):
35                print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36            return load_config(self.config_path)
37
38        def status(self):
39            """Displays status summary of all videos in the database."""
40            # Query total count by status via the public DB read API.
41            rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43            if not rows:
44                print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45            return
46
47            data = [[sport, status, cnt] for sport, status, cnt in rows]
48            print("\n=== Dataset Ingestion Status ===")
49            print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50            print()
51
52        @staticmethod
53        def _annotation_key(sport_str, a):
54            """The per-sport primary-key value of one annotation (for duplicate
detection)."""
55            if sport_str in ("badminton", "pickleball", "tabletennis"):
56                return getattr(a, "rally_number", None)
57            if sport_str == "tennis":
58                return getattr(a, "point_number", None)
59            if sport_str == "cricket":
60                return (getattr(a, "over", None), getattr(a, "ball", None))
61            return None
62
63        def _guard_import(self, sport_str, video_id, annotations, force=False):
64            """Pre-insert safety checks (run BEFORE any DB write, so a rejection is
atomic)."""
65            # (1) Duplicate primary-key guard. int(float()) truncation (e.g. 7 and 7.5 both

```

```

66         # become 7) would otherwise raise IntegrityError mid-insert, leaving a committed
67         # video row with rolled-back annotations. Catch it before touching the DB.
68         keys = [self._annotation_key(sport_str, a) for a in annotations]
69         keys = [k for k in keys if k is not None]
70         seen, dupes = set(), set()
71         for k in keys:
72             if k in seen:
73                 dupes.add(k)
74             seen.add(k)
75         if dupes:
76             print(f"Error: duplicate rally/point identifier(s) {sorted(dupes)} in the
annotations "
77                   f"(often a fractional id like 7.5 truncating to 7).")
78             print(" -> re-run with --normalize, or normalize first: "
79                   "python -m src.cli.normalize_annotations --in <csv> --out clean.csv")
80             sys.exit(1)
81
82         # (2) Golden-label protection ? refuse to silently clobber an existing video's
83         # annotations (filename-derived video_ids collide easily).
84         if self.db.get_video(video_id):
85             try:
86                 n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
87             except Exception:
88                 n = 0
89             if n > 0 and not force:
90                 print(f"Error: video_id '{video_id}' already has {n} annotated
segment(s); "
91                       f"importing would REPLACE them.")
92                 print(" -> pass --force to overwrite, or use a distinct --video-id to
keep both.")
93                 sys.exit(1)
94             if n > 0:
95                 print(f"Warning: --force given ? replacing {n} existing segment(s) for
video_id '{video_id}'.")
96
97         def import_local(self, args):
98             """Manually registers a local video and annotation file into the database."""
99             sport_str = args.sport.lower()
100             if sport_str not in self.config.get("supported_sports", ["badminton"]):
101                 print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
102                 sys.exit(1)
103
104             # Validate video path
105             if not os.path.exists(args.video_path):
106                 print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
107
108             # Validate annotation file
109             if not os.path.exists(args.annotations_path):
110                 print(f"Error: Annotations file '{args.annotations_path}' not found.")
111                 sys.exit(1)
112
113             match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
114             video_id = args.video_id or match_name.lower().replace(" ", "_")
115
116             # Parse license
117             try:
118                 license_type = LicenseType(args.license)
119             except ValueError:
120                 print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
121                 license_type = LicenseType.UNKNOWN
122
123             is_commercial = self.validator.check_license_commercial(license_type)

```

```

124
125     # Optional: normalize a raw CSV annotator export before parsing (drops unusable
126     # rows, rennumbers fractional/duplicate rally ids) so the landmines below can't
127     fire.
128     annotations_path = args.annotations_path
129     if getattr(args, "normalize", False) and
130     annotations_path.lower().endswith(".csv"):
131         from src.cli.normalize_annotations import normalize_file
132         normalized_path = annotations_path + ".normalized.csv"
133         rep = normalize_file(annotations_path, normalized_path)
134         print("=== normalize-annotations ===")
135         print(rep.summary())
136         annotations_path = normalized_path
137
138     # 1. Parse Annotations
139     annotations = []
140     try:
141         annotations = self._parse_local_annotations(sport_str, video_id,
142         annotations_path)
143     except Exception as e:
144         print(f"Error parsing annotations: {e}")
145         sys.exit(1)
146
147     # 2. Run Validation
148     is_valid = True
149     errors = []
150     if sport_str == "badminton":
151         is_valid, errors = self.validator.validate_badminton_rallies(annotations)
152     elif sport_str == "tennis":
153         is_valid, errors = self.validator.validate_tennis_points(annotations)
154     elif sport_str == "cricket":
155         is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
156     elif sport_str == "pickleball":
157         is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
158     elif sport_str == "tabletennis":
159         is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
160
161     if not is_valid:
162         print(f"Error: Annotations failed validation!")
163         for err in errors:
164             print(f" - {err}")
165         sys.exit(1)
166
167     # 2b. Pre-insert safety ? checked BEFORE any DB write, so a rejection never
168     # leaves a half-updated DB (a committed video row with rolled-back annotations).
169     self._guard_import(sport_str, video_id, annotations, force=getattr(args,
170     "force", False))
171
172     # 3. Create Video Metadata
173     video_meta = VideoMetadata(
174         video_id=video_id,
175         sport=SportType(sport_str),
176         video_url=None,
177         local_path=args.video_path,
178         license_type=license_type,
179         license_url=None,
180         is_commercial=is_commercial,
181         status=CurationStatus.CURATED, # Custom local imports bypass staging by
182         default
183         match_name=match_name,
184         fps=args.fps,
185         resolution=args.resolution
186     )
187
188     # 4. Insert into Database

```



```

184 self.db.insert_video(video_meta)
185
186 count = 0
187 if sport_str == "badminton":
188     count = self.db.insert_badminton_rallies(annotations)
189 elif sport_str == "tennis":
190     count = self.db.insert_tennis_points(annotations)
191 elif sport_str == "cricket":
192     count = self.db.insert_cricket_deliveries(annotations)
193 elif sport_str == "pickleball":
194     count = self.db.insert_pickleball_rallies(annotations)
195 elif sport_str == "tabletennis":
196     count = self.db.insert_tabletennis_rallies(annotations)
197
198 print(f"\nSuccessfully imported local video '{match_name}':")
199 print(f" - Video ID: {video_id}")
200 print(f" - Sport: {sport_str}")
201 print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
202 print(f" - Registered {count} annotated segments.\n")
203
204 # Per-video observability report (next to the video). Never raises.
205 from src.reporting import emit_import_report
206 src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
207
208 report_paths = emit_import_report(
209     video_meta_obj=video_meta, annotations=annotations, is_valid=True,
210     errors=[], count=count, sport=sport_str, source_format=src_fmt,
211     fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
212 )
213 if report_paths:
214     print(f" - Observability report: {report_paths.get('technical_md')}")
215     print(f" - Customer summary : {report_paths.get('customer_md')}\n")
216
217 def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
218     """Parses simple JSON or CSV annotations files."""
219     parsed_items = []
220     ext = os.path.splitext(filepath)[1].lower()
221
222     if ext == '.json':
223         with open(filepath, 'r', encoding='utf-8') as f:
224             data = json.load(f)
225
226         if not isinstance(data, list):
227             raise ValueError("JSON file must contain a list of objects.")
228
229         # Uses the same safe casts as the CSV path: a missing/blank required
230         # time raises a clear ValueError (caught by the caller) instead of a
231         # bare KeyError, and optional ints degrade to sensible defaults.
232         for idx, obj in enumerate(data):
233             if sport == "badminton":
234                 parsed_items.append(BadmintonRally(
235                     video_id=video_id,
236                     rally_number=_safe_int(obj.get("rally_number"), idx + 1),
237                     start_time=_safe_float(obj.get("start_time")),
238                     end_time=_safe_float(obj.get("end_time")),
239                     score_before=obj.get("score_before"),
240                     score_after=obj.get("score_after"),
241                     shots_count=_safe_int(obj.get("shots_count"), None),
242                     ending_reason=obj.get("ending_reason"),
243                     last_shot_outcome=obj.get("last_shot_outcome")
244                 ))
245             elif sport == "tennis":
246                 parsed_items.append(TennisPoint(

```

```

246         video_id=video_id,
247         point_number=_safe_int(obj.get("point_number"), idx + 1),
248         start_time=_safe_float(obj.get("start_time")),
249         end_time=_safe_float(obj.get("end_time")),
250         set_score=obj.get("set_score"),
251         game_score=obj.get("game_score"),
252         server=obj.get("server"),
253         ending_type=obj.get("ending_type")
254     ))
255 elif sport == "cricket":
256     parsed_items.append(CricketDelivery(
257         video_id=video_id,
258         over=_safe_int(obj.get("over"), 0),
259         ball=_safe_int(obj.get("ball"), 0),
260         start_time=_safe_float(obj.get("start_time")),
261         end_time=_safe_float(obj.get("end_time")),
262         batsman=obj.get("batsman"),
263         bowler=obj.get("bowler"),
264         runs=_safe_int(obj.get("runs"), None),
265         wicket=bool(obj.get("wicket", False)),
266         extras=_safe_int(obj.get("extras"), 0)
267     ))
268 elif sport == "pickleball":
269     parsed_items.append(PickleballRally(
270         video_id=video_id,
271         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
272         start_time=_safe_float(obj.get("start_time")),
273         end_time=_safe_float(obj.get("end_time")),
274         score_before=obj.get("score_before"),
275         score_after=obj.get("score_after"),
276         shots_count=_safe_int(obj.get("shots_count"), None),
277         ending_reason=obj.get("ending_reason"),
278         last_shot_outcome=obj.get("last_shot_outcome")
279     ))
280 elif sport == "tabletennis":
281     parsed_items.append(TableTennisRally(
282         video_id=video_id,
283         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
284         start_time=_safe_float(obj.get("start_time")),
285         end_time=_safe_float(obj.get("end_time")),
286         score_before=obj.get("score_before"),
287         score_after=obj.get("score_after"),
288         shots_count=_safe_int(obj.get("shots_count"), None),
289         ending_reason=obj.get("ending_reason"),
290         last_shot_outcome=obj.get("last_shot_outcome")
291     ))
292 elif ext == '.csv':
293     with open(filepath, 'r', encoding='utf-8') as f:
294         reader = csv.DictReader(f)
295         # Normalize headers
296         reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
297
298     for idx, row in enumerate(reader):
299         if sport == "badminton":
300             parsed_items.append(BadmintonRally(
301                 video_id=video_id,
302                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
303                 start_time=_safe_float(row.get("start_time")),
304                 end_time=_safe_float(row.get("end_time")),
305                 score_before=row.get("score_before"),
306                 score_after=row.get("score_after"),
307                 shots_count=_safe_int(row.get("shots_count"), None),
308                 ending_reason=row.get("ending_reason"),
309                 last_shot_outcome=row.get("last_shot_outcome")

```

```

310         ))
311     elif sport == "tennis":
312         parsed_items.append(TennisPoint(
313             video_id=video_id,
314             point_number=_safe_int(row.get("point_number"), idx + 1),
315             start_time=_safe_float(row.get("start_time")),
316             end_time=_safe_float(row.get("end_time")),
317             set_score=row.get("set_score"),
318             game_score=row.get("game_score"),
319             server=row.get("server"),
320             ending_type=row.get("ending_type")
321         ))
322     elif sport == "cricket":
323         parsed_items.append(CricketDelivery(
324             video_id=video_id,
325             over=_safe_int(row.get("over"), 0),
326             ball=_safe_int(row.get("ball"), 0),
327             start_time=_safe_float(row.get("start_time")),
328             end_time=_safe_float(row.get("end_time")),
329             batsman=row.get("batsman"),
330             bowler=row.get("bowler"),
331             runs=_safe_int(row.get("runs"), None),
332             wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
333             extras=_safe_int(row.get("extras"), 0)
334         ))
335     elif sport == "pickleball":
336         parsed_items.append(PickleballRally(
337             video_id=video_id,
338             rally_number=_safe_int(row.get("rally_number"), idx + 1),
339             start_time=_safe_float(row.get("start_time")),
340             end_time=_safe_float(row.get("end_time")),
341             score_before=row.get("score_before"),
342             score_after=row.get("score_after"),
343             shots_count=_safe_int(row.get("shots_count"), None),
344             ending_reason=row.get("ending_reason"),
345             last_shot_outcome=row.get("last_shot_outcome")
346         ))
347     elif sport == "tabletennis":
348         parsed_items.append(TableTennisRally(
349             video_id=video_id,
350             rally_number=_safe_int(row.get("rally_number"), idx + 1),
351             start_time=_safe_float(row.get("start_time")),
352             end_time=_safe_float(row.get("end_time")),
353             score_before=row.get("score_before"),
354             score_after=row.get("score_after"),
355             shots_count=_safe_int(row.get("shots_count"), None),
356             ending_reason=row.get("ending_reason"),
357             last_shot_outcome=row.get("last_shot_outcome")
358         ))
359     else:
360         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
361
362     return parsed_items
363
364 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
365     """Download one video at best resolution, retrying transient failures.
366
367     A below-min-height result almost always means YouTube rate-limited the
368     request and served a low fallback (not that the video lacks HD), so we
369     retry with a growing backoff. Returns (DownloadResult, None) on success,
370     or (None, failure_reason) once retries are exhausted. A below-min file is
371     discarded each attempt so it never lands in the DB.
372     """

```

```

373         attempts = max(1, args.retries + 1)
374         reason = "unknown error"
375         for i in range(attempts):
376             result = download_best_video(
377                 url, dest_path,
378                 max_height=max_height, max_size_mb=max_size_mb,
379                 cookies_from_browser=args.cookies_from_browser,
380                 cookies_file=args.cookies,
381                 player_client=args.player_client,
382             )
383             if result.success and (result.height is None or result.height >=
args.min_height):
384                 return result, None
385
386             if not result.success:
387                 reason = result.error or "unknown error"
388             else:
389                 reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
390                         f"YouTube served a low format (rate-limited?)")
391             try:
392                 os.remove(result.path)
393             except OSError:
394                 pass
395
396             if i < attempts - 1:
397                 backoff = args.retry_backoff * (i + 1)
398                 print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in
{backoff:.0f}s...")
399                 time.sleep(backoff)
400         return None, reason
401
402     def fetch(self, args):
403         """(Re)download videos at best available resolution, updating the DB in place.
404
405         This is the in-place upgrade path: rows, rally annotations, and curation
406         status are all preserved ? only the video file plus its `local_path`,
407         `resolution`, and `fps` are refreshed. By default it skips videos that
408         already have a local file at or above `--min-height`, so re-runs are cheap
409         and resumable; `--force` re-fetches everything.
410
411         storage = self.config.get("video_storage", {})
412         local_dir = storage.get("local_dir", "./data/videos")
413         max_download_size_mb = storage.get("max_download_size_mb")
414         max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
415         os.makedirs(local_dir, exist_ok=True)
416
417         commercial_only = not args.include_noncommercial
418         if args.status == "all":
419             statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
420         else:
421             statuses = [CurationStatus(args.status)]
422
423         supported = self.config.get("supported_sports", ["badminton"])
424         sport_strs = [args.sport] if args.sport else supported
425
426         # Collect target videos.
427         videos = []
428         for sport_str in sport_strs:
429             try:
430                 sport = SportType(sport_str.lower())
431             except ValueError:
432                 print(f"Warning: skipping unknown sport '{sport_str}'.")
433                 continue

```

```

434         for status in statuses:
435             videos.extend(self.db.get_videos_by_status(status, sport))
436     if args.video_id:
437         videos = [v for v in videos if v.video_id == args.video_id]
438
439     if not videos:
440         print("\nNo matching videos to fetch.\n")
441         return
442
443     results = [] # (video_id, action, detail)
444     for video in videos:
445         if commercial_only and not video.is_commercial:
446             results.append((video.video_id, "skip", "non-commercial"))
447             continue
448         if not video.video_url or "watch?v=example" in video.video_url:
449             # Either no URL, or the parser's placeholder (the ShuttleSet catalog
450             # had no URL for this match) ? nothing to download.
451             results.append((video.video_id, "skip", "no real source URL"))
452             continue
453
454         dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
455
456         # Skip if we already have a good-enough local file (unless --force).
457         if not args.force and os.path.exists(dest_path):
458             _, height, _ = probe_resolution_fps(dest_path)
459             if height is not None and height >= args.min_height:
460                 results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
461                 continue
462
463         print(f"Fetching {video.video_id} ({video.match_name})...")
464         result, fail_reason = self._fetch_one_with_retry(
465             video.video_url, dest_path, args, max_height, max_download_size_mb)
466         if fail_reason is not None:
467             results.append((video.video_id, "FAIL", fail_reason))
468             if args.sleep:
469                 time.sleep(args.sleep)
470             continue
471
472         # Update only the file-derived fields; preserve everything else (UPSERT
473         # mutates the row in place, leaving rallies intact).
474         video.local_path = result.path
475         if result.resolution_label:
476             video.resolution = result.resolution_label
477         if result.fps:
478             video.fps = result.fps
479         self.db.insert_video(video)
480
481         detail = (f"{result.resolution_label or '?'}"
482                 f"{f' {result.fps:g}fps' if result.fps else ''},
{result.size_mb:.0f} MB")
483         results.append((video.video_id, "fetched", detail))
484         if args.sleep:
485             time.sleep(args.sleep)
486
487     print("\n=== Fetch summary ===")
488     print(tabulate([[vid, action, detail] for vid, action, detail in results],
489                   headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
490     fetched = sum(1 for _, a, _ in results if a == "fetched")
491     failed = [r for r in results if r[1] == "FAIL"]
492     print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _,a,_ in
results if a=='have')}}    "
493           f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}}    Failed :
{len(failed)}}")
494     if failed:

```

```

495         print("Failures (left unchanged in DB):")
496         for vid, _, detail in failed:
497             print(f" - {vid}: {detail}")
498     print()
499
500     # Code/data licenses that can NEVER apply to video footage ? if one of these is
501     # stamped on a remote/broadcast video it is an annotation-vs-video license
502     conflation.
503     _CODE_DATA_LICENSES = {LicenseType.MIT, LicenseType.APACHE_2, LicenseType.BSD}
504
505     def reclassify_licenses(self, args):
506         """Correct annotation-vs-video license conflation in existing rows.
507
508         A third-party video (one with a remote `video_url` ? i.e. broadcast footage like
509         ShuttleSet, whether or not it has since been downloaded to `local_path`) whose
510         `license_type` is a code/data license (MIT/Apache-2.0/BSD) is a conflation: that
511         license covers the ANNOTATIONS, not the footage. This moves the license to
512         `annotation_license`, sets the video `license_type` to Unknown, and recomputes
513         `is_commercial=False` so such videos stop being exported as commercially clean.
514
515         Dry-run by default; pass --apply to persist.
516         """
517         changed = []
518         seen = set()
519         for status in CurationStatus:
520             for video in self.db.get_videos_by_status(status):
521                 if video.video_id in seen:
522                     continue
523                 seen.add(video.video_id)
524                 # The broadcast signal is a third-party video_url; a downloaded copy
525                 # (local_path set) does not change that the FOOTAGE rights are not ours.
526                 is_third_party = bool(video.video_url)
527                 if is_third_party and video.license_type in self._CODE_DATA_LICENSES:
528                     changed.append(video)
529
530         if not changed:
531             print("No conflated rows found ? every remote video already has a video-
532             footage license.")
533             return
534
535         print(f'{"APPLYING" if args.apply else "DRY-RUN"} ? {len(changed)} conflated
536         row(s):')
537         for v in changed:
538             print(f" {v.video_id:40s} {v.license_type.value} (video) ->
539             annotation_license; "
540             f"video license -> Unknown; is_commercial {v.is_commercial} -> False")
541             if args.apply:
542                 v.annotation_license = v.annotation_license or v.license_type
543                 v.license_type = LicenseType.UNKNOWN
544                 v.is_commercial =
545                 self.validator.check_license_commercial(v.license_type)
546                 self.db.insert_video(v)
547
548         if args.apply:
549             print("\nApplied. Re-run `curate export` to regenerate the manifest without
550             these videos.")
551         else:
552             print("\nDry-run only. Re-run with --apply to persist, then `curate
553             export`.")
554
555     def export_eval_set(self, args):
556         """Export curated annotations as indexer-ready eval/training sets.
557
558         Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
559         indexer's `start,end` ground-truth format (decimal seconds), plus a single

```

```

553     `<out_dir>/manifest.json` that pairs each CSV with its video file. The
554     manifest is the contract the downstream eval/training driver reads: the
555     indexer keys videos by *file MD5*, not by our `video_id`, so it must run
556     against the exact `local_path`/`video_url` recorded here.
557
558     By default only CURATED + commercially-safe videos are exported, mirroring
559     the license gate already enforced elsewhere in the toolchain.
560     """
561     out_dir = args.out_dir
562     commercial_only = not args.include_noncommercial
563     statuses = [CurationStatus.CURATED]
564     if args.include_staging:
565         statuses.append(CurationStatus.STAGING)
566
567     supported = self.config.get("supported_sports", ["badminton"])
568     sport_strs = [args.sport] if args.sport else supported
569
570     manifest = []
571     skipped_noncomm = 0
572     skipped_empty = 0
573
574     for sport_str in sport_strs:
575         try:
576             sport = SportType(sport_str.lower())
577         except ValueError:
578             print(f"Warning: skipping unknown sport '{sport_str}'.")
579             continue
580
581         videos = []
582         for status in statuses:
583             videos.extend(self.db.get_videos_by_status(status, sport))
584
585         for video in videos:
586             if commercial_only and not video.is_commercial:
587                 skipped_noncomm += 1
588                 continue
589
590             intervals = self.db.get_segment_intervals(video.video_id, sport)
591             if not intervals:
592                 skipped_empty += 1
593                 continue
594
595             sport_dir = os.path.join(out_dir, sport.value)
596             os.makedirs(sport_dir, exist_ok=True)
597             csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
598             with open(csv_path, "w", newline="", encoding="utf-8") as f:
599                 writer = csv.writer(f)
600                 writer.writerow(["start", "end"])
601                 for start, end in intervals:
602                     writer.writerow([start, end])
603
604             manifest.append({
605                 "video_id": video.video_id,
606                 "sport": sport.value,
607                 "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
608                 "local_path": video.local_path,
609                 "video_url": video.video_url,
610                 "match_name": video.match_name,
611                 "fps": video.fps,
612                 "resolution": video.resolution,
613                 "license_type": video.license_type.value,
614                 "is_commercial": video.is_commercial,
615                 "status": video.status.value,
616                 "num_segments": len(intervals),
617             })

```

```

618
619     os.makedirs(out_dir, exist_ok=True)
620     manifest_doc = {
621         "commercial_only": commercial_only,
622         "statuses": [s.value for s in statuses],
623         "total_videos": len(manifest),
624         "total_segments": sum(m["num_segments"] for m in manifest),
625         "eval_sets": manifest,
626     }
627     manifest_path = os.path.join(out_dir, "manifest.json")
628     with open(manifest_path, "w", encoding="utf-8") as f:
629         json.dump(manifest_doc, f, indent=2)
630
631     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
632     if manifest:
633         summary = [[m["sport"], m["video_id"], m["num_segments"],
634                     "YES" if m["is_commercial"] else "NO",
635                     os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
636                     for m in manifest]
637         print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
638         print(f"\nVideos exported : {len(manifest)}")
639         print(f"Total segments : {manifest_doc['total_segments']}")
640         if skipped_noncomm:
641             print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
642         if skipped_empty:
643             print(f"Skipped (no annotations) : {skipped_empty}")
644         print(f"Manifest : {manifest_path}\n")
645
646     def prep_annotation_proxy(self, args):
647         """Re-encode a video into a frame-identical annotation proxy (ADR-002).
648
649         The proxy is what the annotation vendor receives: downscaled and
650         scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
651         but with the source's EXACT frame count / fps / timeline, so frame N
652         on the proxy is frame N on the original and returned labels apply to
653         the original unchanged. Parity is verified after the encode; a proxy
654         that fails verification is deleted, never shared. Provenance (source +
655         proxy MD5s) is appended to the proxy manifest ? downstream processing
656         (import-local, export, the indexer) keeps running on the ORIGINAL file.
657         """
658         from src.core import proxy as proxylib
659
660         src = args.video_path
661         if not os.path.exists(src):
662             print(f"Error: video '{src}' not found.")
663             sys.exit(1)
664
665         stem = os.path.basename(src).split('.')[0]
666         # Mirror import-local's video_id derivation so the manifest row and a
667         # later DB registration of the same source agree on the key.
668         video_id = args.video_id or stem.lower().replace(" ", "_")
669         out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
670         if os.path.abspath(out) == os.path.abspath(src):
671             print("Error: --out must differ from --video-path.")
672             sys.exit(1)
673         if os.path.exists(out) and not args.force:
674             print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
675             sys.exit(1)
676
677         try:
678             src_info = proxylib.probe_video(src)
679         except proxylib.ProxyError as e:

```



```

680         print(f"Error: {e}")
681         sys.exit(1)
682     if src_info.avg_fps is None:
683         print(f"Error: could not determine fps of '{src}' ? the frame<->time "
684               f"contract needs it.")
685         sys.exit(1)
686     if proxylib.is_vfr(src_info) and not args.allow_vfr:
687         print(f"Error: '{src}' looks variable-frame-rate "
688               f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
689         print(" The vendor pipeline recomputes times as frame/fps, which is only "
690               "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
691               "proceed anyway (timestamps stay identical to the original, but "
692               "frame/fps seconds are approximate for BOTH files).")
693         sys.exit(1)
694
695     os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
696     cmd = proxylib.build_proxy_command(src, out, height=args.height,
697                                       crf=args.crf, gop=args.gop,
698                                       preset=args.preset)
699     print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
700           f"fps {src_info.avg_fps:g} preserved)...")
701     try:
702         proxylib.run_ffmpeg(cmd)
703         proxy_info = proxylib.probe_video(out)
704     except proxylib.ProxyError as e:
705         print(f"Error: {e}")
706         sys.exit(1)
707
708     errors = proxylib.verify_parity(src_info, proxy_info)
709     if errors:
710         print("Error: proxy FAILED frame<->time parity verification:")
711         for err in errors:
712             print(f" - {err}")
713         try:
714             os.remove(out)
715             print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
716                   f"poisons golden labels).")
717         except OSError:
718             print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
719                   f"share it with the vendor.")
720         sys.exit(1)
721
722     print("Hashing source + proxy (provenance)...")
723     source_md5 = proxylib.md5_file(src)
724     proxy_md5 = proxylib.md5_file(out)
725
726     from datetime import datetime, timezone
727     proxylib.append_manifest_row(args.manifest, {
728         "video_id": video_id,
729         "source_path": os.path.abspath(src),
730         "source_md5": source_md5,
731         "proxy_path": os.path.abspath(out),
732         "proxy_md5": proxy_md5,
733         "fps": f"{src_info.avg_fps:g}",
734         "frame_count": src_info.nb_frames,
735         "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None
736     else "",
737         "source_resolution": src_info.resolution,
738         "proxy_resolution": proxy_info.resolution,
739         "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
740                             f"preset={args.preset} height<={args.height} "
741                             f"fps_mode=passthrough audio=aac160k"),
742         "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
743     })

```

```

744         src_mb = os.path.getsize(src) / (1024 * 1024)
745         out_mb = os.path.getsize(out) / (1024 * 1024)
746         print("\n=== Annotation proxy ready ===")
747         print(f" Video ID      : {video_id}")
748         print(f" Source        : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
{source_md5})")
749         print(f" Proxy          : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5
{proxy_md5})")
750         print(f" Parity          : VERIFIED ? {src_info.nb_frames} frames @
{src_info.avg_fps:g} fps on both")
751         print(f" Manifest       : {args.manifest}")
752         print("\nNext:")
753         print(" 1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
754             "box of every rally on the exact contact/dead frame).")
755         print(" 2. On delivery, convert-cvat may use --video-path with either file "
756             "(identical fps/frame count by construction).")
757         print(" 3. import-local / export keep pointing at the ORIGINAL file ? the "
758             "indexer keys videos by its MD5.\n")
759
760     def convert_cvat(self, args):
761         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
762
763         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
764         attributes) on the ingestion side: each rally's time is recomputed from
765         its frame range at the video's true fps, and content defects (overlaps,
766         zero-duration, off-vocabulary endings, coarse sampling) are written to an
767         issues report rather than silently "fixed". The emitted CSV imports via
768         `import-local`; any blocker issue must be resolved first, since the
769         importer's validator rejects overlapping / zero-duration rallies.
770         """
771         from src.parsers.cvat.rally_cvat import (
772             convert, write_canonical_csv, render_issue_report,
773         )
774         try:
775             result = convert(args.input, fps=args.fps,
776                             video_path=args.video_path, step=args.step)
777         except (FileNotFoundError, ValueError) as e:
778             print(f"Error: {e}")
779             sys.exit(1)
780
781         if not result.records:
782             print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
783                 f"Likely the wrong job was exported or the rally_number attribute was
"
784                 f"renamed (boxes without a matched rally_number are skipped).")
785             sys.exit(1)
786
787         write_canonical_csv(result.records, args.out)
788
789         # Observability report next to the canonical CSV. Never raises.
790         from src.reporting import emit_delivery_report
791         src_name = os.path.splitext(os.path.basename(args.input))[0]
792         dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
793         emit_delivery_report(
794             out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
795             records=result.records, issues=result.issues, fps=result.fps,
796             video_path=args.video_path,
797             video_duration_s=dur, source_format="cvat", step=result.step,
798         )
799         report = render_issue_report(result, os.path.basename(args.input))
800         if args.issues:
801             with open(args.issues, "w", encoding="utf-8") as f:
802                 f.write(report)

```

```

803         print(report)
804         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
805         if result.n_blockers:
806             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
807             f"this file until they are resolved (by the vendor or manual
review).")
808         else:
809             print("\n No blockers. Next:")
810             print(f"    python -m src.cli.curate import-local --sport <sport> "
811                   f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
812
813     def validate_delivery(self, args):
814         """Run the full validation suite on a rally delivery and emit a report.
815
816         Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
817         CSV (--csv). Writes, into --out-dir:
818             <id>_report.md    human report (scoreboard + issues + rallies to review)
819             <id>_report.json  machine summary (for CI / a bulk-ingest loop)
820             <id>_review.csv   the annotatable manual-verification sheet
821         Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
822         """
823         import json
824         from src.parsers.cvat.rally_cvat import (
825             convert, load_canonical_csv, detect_issues,
826             build_summary, render_validation_md, write_review_csv,
827         )
828         vid = args.video_id or "delivery"
829         fps = None
830         step = None
831         stop_frame = None
832         if args.input:
833             try:
834                 result = convert(args.input, fps=args.fps, video_path=args.video_path)
835             except (FileNotFoundError, ValueError) as e:
836                 print(f"Error: {e}")
837                 sys.exit(1)
838             records, issues, fps = result.records, result.issues, result.fps
839             step = result.step
840             stop_frame = result.meta.get("stop_frame")
841             if not records:
842                 print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
843                       f"The wrong job was exported or the rally_number attribute was
renamed. "
844                       f"GATE: BLOCKED.")
845                 sys.exit(2)
846             elif args.csv:
847                 try:
848                     records = load_canonical_csv(args.csv)
849                 except FileNotFoundError as e:
850                     print(f"Error: {e}")
851                     sys.exit(1)
852                 if not records:
853                     print(f"Error: no usable rally rows in {args.csv}")
854                     sys.exit(1)
855                 issues = detect_issues(records)
856             else:
857                 print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
858                 sys.exit(1)
859
860         os.makedirs(args.out_dir, exist_ok=True)
861         summary = build_summary(records, issues, source=vid, fps=fps)
862         md = render_validation_md(records, issues, source=vid)
863         json_path = os.path.join(args.out_dir, f"{vid}_report.json")

```

```

864         md_path = os.path.join(args.out_dir, f"{vid}_report.md")
865         review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
866         with open(json_path, "w", encoding="utf-8") as f:
867             json.dump(summary, f, indent=2)
868         with open(md_path, "w", encoding="utf-8") as f:
869             f.write(md)
870         write_review_csv(records, issues, review_path)
871
872         # Observability report (envelope JSON + technical md + customer summary),
873         # colocated with the existing artifacts. Never raises.
874         from src.reporting import emit_delivery_report
875         duration_s = (stop_frame / fps) if (stop_frame and fps) else None
876         report_paths = emit_delivery_report(
877             out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
878             video_path=args.video_path, video_duration_s=duration_s,
879             source_format=("cvat" if args.input else "csv"), step=step,
880         )
881
882         print(md)
883         print(f"\nWrote:\n {md_path}\n {json_path}\n {review_path}")
884         if report_paths:
885             print(f" {report_paths.get('technical_md')}\n
{report_paths.get('customer_md')}\n {report_paths.get('json')}")
886         if summary["n_blockers"]:
887             print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
888             sys.exit(2)
889         print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
890
891     def curate(self):
892         """Interactive loop to review and curate staging video annotations."""
893         staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
894         if not staging_videos:
895             print("\nNo staging video annotations to review.\n")
896             return
897
898         print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
899         table_data = []
900         for idx, video in enumerate(staging_videos):
901             table_data.append([
902                 idx + 1,
903                 video.video_id,
904                 video.match_name,
905                 video.sport.value,
906                 video.license_type.value,
907                 "YES" if video.is_commercial else "NO"
908             ])
909
910         print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
911         print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))
912
913         choice = input("> ").strip()
914         if choice.lower() == 'q':
915             return
916
917         try:
918             idx = int(choice) - 1
919             if idx < 0 or idx >= len(staging_videos):
920                 print("Invalid choice.")
921                 return
922             self._curate_video_interactive(staging_videos[idx])
923         except ValueError:
924             print("Invalid input.")

```

```

925
926     def _curate_video_interactive(self, video: VideoMetadata):
927         """Displays details of a specific staging video and prompts for curation
928         action."""
929
930         # Load and count events
931         rallies = []
932         pts = []
933         delivs = []
934         if video.sport == SportType.BADMINTON:
935             rallies = self.db.get_badminton_rallies(video.video_id)
936         elif video.sport == SportType.PICKLEBALL:
937             rallies = self.db.get_pickleball_rallies(video.video_id)
938         elif video.sport == SportType.TABLE_TENNIS:
939             rallies = self.db.get_tabletennis_rallies(video.video_id)
940         elif video.sport == SportType.TENNIS:
941             pts = self.db.get_tennis_points(video.video_id)
942         elif video.sport == SportType.CRICKET:
943             delivs = self.db.get_cricket_deliveries(video.video_id)
944
945         while True:
946             print("\n===== REVIEWING VIDEO =====")
947             print(f"Match Name : {video.match_name}")
948             print(f"Video ID : {video.video_id}")
949             print(f"Sport : {video.sport.value}")
950             print(f"Video URL : {video.video_url or 'N/A'}")
951             print(f"Local Path : {video.local_path or 'N/A'}")
952             print(f"License : {video.license_type.value} ({video.license_url or 'no
953             URL'})")
954             print(f"Commercial OK: {video.is_commercial}")
955
956             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
957             SportType.TABLE_TENNIS):
958                 print(f"Total Rallies: {len(rallies)}")
959                 if rallies:
960                     print("Sample rallies:")
961                     sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
962                     r.score_after, r.shots_count] for r in rallies[:5]]
963                     print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
964                     "Shots"], tablefmt="simple"))
965                 elif video.sport == SportType.TENNIS:
966                     print(f"Total Points : {len(pts)}")
967                     if pts:
968                         print("Sample points:")
969                         sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
970                         p.game_score] for p in pts[:5]]
971                         print(tabulate(sample_data, headers=["Point #", "Duration",
972                         "Score"], tablefmt="simple"))
973                 elif video.sport == SportType.CRICKET:
974                     print(f"Total Balls : {len(delivs)}")
975                     if delivs:
976                         print("Sample deliveries:")
977                         sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
978                         {d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
979                         print(tabulate(sample_data, headers=["Over.Ball", "Duration",
980                         "Runs", "Wicket"], tablefmt="simple"))
981
982             print("\nChoose Curation Action:")
983             print(" [a] Approve (promote to CURATED)")
984             print(" [r] Reject (delete from database)")
985             print(" [p] Play a rally/point segment in browser")
986             print(" [k] Keep in Staging (do nothing and go back)")
987
988             action = input("> ").strip().lower()
989             if action == 'a':

```

```

981         self.db.update_status(video.video_id, CurationStatus.CURATED)
982         print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
983         break
984     elif action == 'r':
985         self.db.delete_video(video.video_id)
986         print(f"Successfully rejected and deleted '{video.match_name}'.\n")
987         break
988     elif action == 'k':
989         print("Curation state unchanged.\n")
990         break
991     elif action == 'p':
992         self._play_segment_interactive(video, rallies, pts, delivs)
993     else:
994         print("Invalid action selected.")
995
996     def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
997         import webbrowser
998
999         # Get start/end based on user input
1000         start_time = None
1001         end_time = None
1002         label = "segment"
1003
1004         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
1005             if not rallies:
1006                 print("No rallies to play.")
1007                 return
1008             idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
1009             try:
1010                 rally_num = int(idx_str)
1011                 rally = next((r for r in rallies if r.rally_number == rally_num), None)
1012                 if rally:
1013                     start_time = rally.start_time
1014                     end_time = rally.end_time
1015                     label = f"Rally {rally_num}"
1016             except ValueError:
1017                 pass
1018         elif video.sport == SportType.TENNIS:
1019             if not pts:
1020                 print("No points to play.")
1021                 return
1022             idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
1023             try:
1024                 pt_num = int(idx_str)
1025                 pt = next((p for p in pts if p.point_number == pt_num), None)
1026                 if pt:
1027                     start_time = pt.start_time
1028                     end_time = pt.end_time
1029                     label = f"Point {pt_num}"
1030             except ValueError:
1031                 pass
1032         elif video.sport == SportType.CRICKET:
1033             if not delivs:
1034                 print("No deliveries to play.")
1035                 return
1036             over_str = input("Enter Over number: ").strip()
1037             ball_str = input("Enter Ball number: ").strip()
1038             try:
1039                 o_num = int(over_str)
1040                 b_num = int(ball_str)
1041                 d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
1042                 if d:

```

```

1043         start_time = d.start_time
1044         end_time = d.end_time
1045         label = f"Over {o_num} Ball {b_num}"
1046     except ValueError:
1047         pass
1048
1049     if start_time is None or end_time is None:
1050         print("Segment not found or invalid input.")
1051         return
1052
1053     print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
1054
1055     # Check video location
1056     play_url = None
1057     if video.local_path and os.path.exists(video.local_path):
1058         abs_path = os.path.abspath(video.local_path)
1059         # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
1060         play_url = f"file:/// {abs_path.replace(os.sep,
1061         '/'')}#t={start_time},{end_time}"
1062     elif video.video_url:
1063         # YouTube URL: seek to start time
1064         if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
1065             t_sec = int(start_time)
1066             sep = "&" if "?" in video.video_url else "?"
1067             play_url = f"{video.video_url}{sep}t={t_sec}s"
1068         else:
1069             play_url = video.video_url
1070
1071     if play_url:
1072         print(f"Opening browser: {play_url}")
1073         webbrowser.open(play_url)
1074     else:
1075         print("Error: No local video file or remote URL available to play.")
1076
1077 def main():
1078     parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
1079     subparsers = parser.add_subparsers(dest="command")
1080
1081     # Status command
1082     subparsers.add_parser("status", help="Display summary status of sports metadata
database")
1083
1084     # Curate command
1085     subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
1086
1087     # Import-local command
1088     import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
1089     import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
1090     import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
1091     import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
1092     import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
1093     import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
1094     import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
1095     import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
1096     import_parser.add_argument("--resolution", default="1080p", help="Resolution of the

```

```

video (e.g., 1080p, 720p)")
1097     import_parser.add_argument("--force", action="store_true",
1098                                     help="Overwrite an existing video's annotations (default:
refuse, to protect golden labels)")
1099     import_parser.add_argument("--normalize", action="store_true",
1100                                 help="Run normalize-annotations on a CSV input first
(drops unusable rows, "
1101                                     "renumbers fractional/duplicate rally ids 1..N by
start time)")
1102
1103     # Fetch command ? (re)download videos at best resolution, update DB in place
1104     fetch_parser = subparsers.add_parser(
1105         "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
1106     fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
1107     fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
1108     fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
1109                                     help="Which curation statuses to fetch (default: all)")
1110     fetch_parser.add_argument("--min-height", type=int, default=720,
1111                                 help="Skip videos already at/above this height unless
--force (default: 720)")
1112     fetch_parser.add_argument("--max-height", type=int, default=None,
1113                                 help="Cap download resolution (e.g. 1080); default: best
available")
1114     fetch_parser.add_argument("--force", action="store_true",
1115                                 help="Re-download even if a good-enough local file already
exists")
1116     fetch_parser.add_argument("--include-noncommercial", action="store_true",
1117                                 help="Include videos whose license is not commercially
safe (default: commercial only)")
1118     fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
1119                                 help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
1120                                     "A Premium login gives reliable best-resolution
downloads. "
1121                                     "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
1122     fetch_parser.add_argument("--cookies", metavar="FILE",
1123                                 help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
1124     fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
1125                                 help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
1126                                     "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
1127     fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
1128                                 help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
1129     fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
1130                                 help="Retries per video when a download fails or returns a
low format (default: 2)")
1131     fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
1132                                 help="Base backoff between retries; grows linearly per
attempt (default: 20)")
1133
1134     # Prep-annotation-proxy command ? vendor-handoff re-encode (ADR-002)
1135     prep_parser = subparsers.add_parser(
1136         "prep-annotation-proxy",
1137         help="Re-encode a video into a light, frame-identical annotation proxy for "
1138             "vendor handoff (ADR-002): downscaled + scrub-friendly, same frame "

```



```

1139         "count/fps/timeline, parity-verified, provenance recorded")
1140     prep_parser.add_argument("--video-path", required=True,
1141                             help="The ORIGINAL source video (stays the system-of-record
file)")
1142     prep_parser.add_argument("--out",
1143                             help="Proxy output path (default:
data/videos/proxies/<stem>_proxy.mp4)")
1144     prep_parser.add_argument("--video-id",
1145                             help="Manifest key (default: derived from the filename,
like import-local)")
1146     prep_parser.add_argument("--height", type=int, default=1080,
1147                             help="Max proxy height; never upscales (default: 1080 ? "
1148                                 "720 risks losing the table-tennis ball)")
1149     prep_parser.add_argument("--crf", type=int, default=19,
1150                             help="x264 quality, lower=better (default: 19)")
1151     prep_parser.add_argument("--gop", type=int, default=15,
1152                             help="Max keyframe interval in frames; small = snappy "
1153                                 "frame-stepping (default: 15)")
1154     prep_parser.add_argument("--preset", default="medium",
1155                             help="x264 speed/size preset (default: medium)")
1156     prep_parser.add_argument("--manifest", default=os.path.join("data",
"annotation_proxy_manifest.csv"),
1157                             help="Provenance CSV to append to (default:
data/annotation_proxy_manifest.csv; "
1158                                 "commit it ? it is the proxy<->original mapping of
record)")
1159     prep_parser.add_argument("--allow-vfr", action="store_true",
1160                             help="Proceed on a variable-frame-rate source (frame/fps "
1161                                 "seconds are approximate; default: refuse)")
1162     prep_parser.add_argument("--force", action="store_true",
1163                             help="Overwrite an existing proxy file")
1164
1165     # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
1166     cvat_parser = subparsers.add_parser(
1167         "convert-cvat",
1168         help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
1169     cvat_parser.add_argument("--input", required=True,
1170                             help="Path to the CVAT XML export (image- or track-mode)")
1171     cvat_parser.add_argument("--out", required=True,
1172                             help="Path to write the canonical per-rally CSV (import-
local input)")
1173     cvat_parser.add_argument("--video-path",
1174                             help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
1175     cvat_parser.add_argument("--fps", type=float,
1176                             help="Override fps instead of probing the video")
1177     cvat_parser.add_argument("--step", type=int,
1178                             help="Frame sampling step (default: read from CVAT meta,
else inferred)")
1179     cvat_parser.add_argument("--issues",
1180                             help="Path to also write the human-readable issues report")
1181
1182     # Validate-delivery command ? run the full check suite + emit a structured report
1183     vd_parser = subparsers.add_parser(
1184         "validate-delivery",
1185         help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
1186     vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
1187     vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
1188     vd_parser.add_argument("--video-path",
1189                             help="Source video; fps via ffprobe (when using --input)")
1190     vd_parser.add_argument("--fps", type=float, help="Override fps (when using

```

```

--input)")
1191     vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
1192     vd_parser.add_argument("--out-dir", default="data/validation",
1193                             help="Where to write the report + review sheet")
1194
1195     # Export command
1196     export_parser = subparsers.add_parser(
1197         "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
1198     export_parser.add_argument("--out-dir", default="data/eval_export",
1199                               help="Output directory for per-video CSVs and
manifest.json")
1200     export_parser.add_argument("--sport",
1201                               help="Restrict to a single sport (default: all
supported_sports)")
1202     export_parser.add_argument("--include-staging", action="store_true",
1203                               help="Also export videos still in staging (default:
curated only)")
1204     export_parser.add_argument("--include-noncommercial", action="store_true",
1205                               help="Include videos whose license is not commercially
safe (default: commercial only)")
1206
1207     # Reclassify-licenses command (one-shot corrective maintenance)
1208     reclass_parser = subparsers.add_parser(
1209         "reclassify-licenses",
1210         help="Fix annotation-vs-video license conflation (e.g. MIT stamped on broadcast
video)")
1211     reclass_parser.add_argument("--apply", action="store_true",
1212                                help="Persist changes (default: dry-run preview only)")
1213
1214     args = parser.parse_args()
1215     cli = SportsDataCLI()
1216
1217     if args.command == "status":
1218         cli.status()
1219     elif args.command == "reclassify-licenses":
1220         cli.reclassify_licenses(args)
1221     elif args.command == "curate":
1222         cli.curate()
1223     elif args.command == "import-local":
1224         cli.import_local(args)
1225     elif args.command == "prep-annotation-proxy":
1226         cli.prep_annotation_proxy(args)
1227     elif args.command == "convert-cvat":
1228         cli.convert_cvat(args)
1229     elif args.command == "validate-delivery":
1230         cli.validate_delivery(args)
1231     elif args.command == "export":
1232         cli.export_eval_set(args)
1233     elif args.command == "fetch":
1234         cli.fetch(args)
1235     else:
1236         # Default: print help
1237         parser.print_help()
1238
1239     if __name__ == "__main__":
1240         main()
1241

```

5. assistant (2026-06-12T10:44:53.346Z)

Now let me check the ADR, docs, and tests for what sources are anticipated and how the duration fallback is treated.

6. user (2026-06-12T10:44:54.961Z)

76:## ADR-002 ? Annotation proxies are generated in-repo (`curate prep-annotation-proxy`), not by a one-off tool

77-

78-**Status:** Accepted · **Date:** 2026-06-12 (owner-ratified)

79-

80-### Context

81-The Roora pilot showed the vendor annotates in CVAT directly on our **4K / 59.94 fps** originals and copes with the decode cost by sampling **every 30th frame**. The result (pickleball + table-tennis review, 2026-06-10): **every rally boundary off by ~0.5 s**, up to **~2 s**, against the **±0.3 s** bar (Brief §4) ? all 24 boundaries sit exactly on the 30-frame grid (~0.5 s at 59.94 fps), so they **structurally cannot** meet spec. The same grid was flagged **[HIGH]** in the earlier badminton round (`data/roora\_badminton\_issues.txt`); it is their standing workflow, not a one-off lapse.

88-

89-**Phase B** makes this untenable, not just suboptimal: **~12?15 videos per sport** at **~30 min each** (Brief §3). A 30-minute 4K/59.94 file is **>100k frames**; frame-accurate annotation (frame-step 1) on 4K originals at that scale is not realistic for the vendor. A lightweight **annotation proxy** (downscaled, scrub-friendly re-encode of the identical timeline) is therefore close to a **precondition for Phase B**, not an optimization.

94-

95-The pipeline is proxy-friendly by construction: annotations join by the human-assigned **video_id** (not file MD5), and **convert-cvat** recomputes all times as **frame / fps**, ignoring vendor time attributes. The Brief's "do not re-encode" rule binds the **vendor** (timeline + checksum identity of the file they annotate), not us ? provided downstream processing always runs on the **original** file (see **PIPELINE_ROLE.md**: the indexer keys by MD5).

101-

102-### Decision

103-**We pre-process every video before vendor handoff into an annotation proxy, and the generator lives in this repo as a `curate prep-annotation-proxy` subcommand** ? not a standalone script, and not in the indexer.

106-

107-### Proxy contract (invariants the tool must enforce)

108- **Same fps as the source** ? never reduce it. At a lower fps every CVAT frame index changes meaning, and the vendor's 30-frame sampling habit at 30 fps would become a **1.0 s** grid ? twice as coarse. Same frame count, no trimming, no leading padding ? proxy seconds map 1:1 onto the original.

112- **Downscale 4K ? 1080p** (not 720p: table-tennis ball visibility), CRF ~18?20, **dense keyframes** (**-g 15**) so frame-stepping/back-scrubbing is fast, **audio kept** (contact sound is the annotator's best boundary cue).

115- **Verification built in:** assert proxy frame count == original frame count and durations match (also surfaces VFR sources, where **frame / fps** is already unsafe).

117- **Provenance:** emit/append the intake-manifest row ? **video_id**, fps, frame count, **original MD5 + proxy MD5** ? so the proxy?original mapping is durable and downstream processing provably targets the original.

120-

121-### Rationale (why the collector, why not a loose script)

122- **This repo owns the vendor flow end-to-end** (`docs/annotation/`, intake manifest, **convert-cvat** ? **validate-delivery** ? **import-local**); proxy prep is simply command #0 of that existing pipeline, and it already shells out to ffprobe.

125- **The ffmpeg invocation is the trivial 20%**. The durable value is the parity verification + manifest provenance ? exactly what one-off scripts skip. We have precedent for the failure mode: unfilled intake-manifest TODOs, eval artifacts lost in **%TEMP%**, and indexer predictions once keyed under a **downscaled copy's** MD5.

129- **Phase-B frequency:** ~40 runs across sessions/operators ? needs tests + CI, i.e. repo-resident code.

131- **Rejected ? standalone script:** acceptable stopgap for an immediate redelivery, but no durable verification/provenance. **Rejected ? indexer:** it has the re-encode primitive (**extract_video_chunk**) but is the product pipeline; the vendor handoff workflow, manifest, and ingestion all live here.

135-

136-### Consequences / follow-ups

7. user (2026-06-12T10:44:56.139Z)

```
68-     assert is_vfr(ProbeInfo(avg_fps=None, r_fps=30.0))
69-     assert is_vfr(ProbeInfo(avg_fps=30.0, r_fps=None))
70-
71-
72-def _info(frames=5737, fps=59.94006, dur=95.72, w=3840, h=2160):
73-     return ProbeInfo(width=w, height=h, avg_fps=fps, r_fps=fps,
74-                       nb_frames=frames, duration=dur)
75-
76-
77-def test_verify_parity_ok_and_tolerates_audio_priming():
78-     src = _info()
79-     # Same frames/fps; container a hair longer (AAC priming) ? still parity.
80-     proxy = _info(dur=95.76, w=1920, h=1080)
81-     --
90-     errors = verify_parity(_info(fps=59.94006), _info(fps=29.97003, w=1920, h=1080))
91-     assert any("fps changed" in e for e in errors)
92-
93-
94-def test_verify_parity_catches_trim():
95-     errors = verify_parity(_info(dur=95.72), _info(dur=94.20, w=1920, h=1080))
96-     assert any("duration changed" in e for e in errors)
97-
98-
99-def test_verify_parity_loud_on_unknown_identity():
100-     errors = verify_parity(ProbeInfo(), ProbeInfo())
101-     assert len(errors) == 3 # frames, fps, duration all unprovable
102-
103-
104-def test_md5_file(tmp_path):
105-     p = tmp_path / "f.bin"
106-     p.write_bytes(b"hello world")
107-     assert md5_file(str(p)) == "5eb63bbbe01eeed093cb22bb8f5acdc3"
108-     --
124-
125-__FFPROBE_JSON = """
126-{"
127-  "streams": [{
128-    "width": 3840, "height": 2160,
129-    "r_frame_rate": "60000/1001", "avg_frame_rate": "60000/1001",
130-    "duration": "95.720000", "nb_read_packets": "5737"
131-  }],
132-  "format": {"duration": "95.768000"}
133-}
134-"""
135-
136-
137-def test_probe_video_parses_ffprobe_json(tmp_path):
138-     p = tmp_path / "v.mp4"
139-     --
141-     mock.patch("src.core.proxy.subprocess.run",
142-               return_value=SimpleNamespace(stdout=__FFPROBE_JSON)):
143-         info = probe_video(str(p))
144-         assert (info.width, info.height) == (3840, 2160)
145-         assert info.nb_frames == 5737
146-         assert abs(info.avg_fps - 59.94006) < 1e-4
147-         assert info.duration == pytest.approx(95.72) # stream duration preferred
148-         assert info.resolution == "3840x2160"
149-
150-
151-def test_probe_video_requires_ffprobe(tmp_path):
152-     p = tmp_path / "v.mp4"
153-     p.write_bytes(b"x")
154-     --
```

```

253-
254-
255-def test_prep_proxy_refuses_vfr_without_flag(cli, tmp_path):
256-     args = _prep_args(tmp_path)
257-     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
258-     vfr_info = ProbeInfo(width=3840, height=2160, avg_fps=59.2, r_fps=60.0,
259-                           nb_frames=100, duration=1.69)
260-     with mock.patch("src.core.proxy.probe_video", return_value=vfr_info):
261-         with pytest.raises(SystemExit) as exc:
262-             cli.prep_annotation_proxy(args)
263-     assert exc.value.code == 1
264-     assert not os.path.exists(args.out)
265-
266-
267-def test_prep_proxy_allow_vfr_proceeds(cli, tmp_path):
268-     args = _prep_args(tmp_path, allow_vfr=True)
269-     (tmp_path / "Match One.mp4").write_bytes(b"original-bytes")
270-     vfr = ProbeInfo(width=3840, height=2160, avg_fps=59.2, r_fps=60.0,
271-                     nb_frames=100, duration=100 / 59.2)
272-     vfr_proxy = ProbeInfo(width=1920, height=1080, avg_fps=59.2, r_fps=60.0,
273-                           nb_frames=100, duration=100 / 59.2)
274-
275-     def fake_ffmpeg(cmd):
276-         with open(cmd[-1], "wb") as f:
277-             f.write(b"proxy-bytes")
278-
279-     with mock.patch("src.core.proxy.probe_video", side_effect=[vfr, vfr_proxy]), \

```

8. assistant (2026-06-12T10:45:25.240Z)

The mechanism checks out so far. Let me check whether MKV/TS sources are plausible in this repo's scope, and read the new docs.

9. user (2026-06-12T10:45:26.788Z)

```

.gitignore:49:# ---- Per-video observability reports (written next to videos / inputs at
runtime) ----
.github\workflows\ci.yml:8:jobs:
pyproject.toml:28:# Per-video observability reports (SOFT dependency: ingestion degrades to a
no-op
pyproject.toml:31:#   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
pyproject.toml:33:#   pip install -e ../sports-obsreport --no-build-isolation
pyproject.toml:35:   "obsreport>=0.1.3",
docs\INGESTION_PIPELINE.md:151:"goes missing" downstream, check its `is_commercial` flag first.
(The observability
docs\INGESTION_PIPELINE.md:154:### What is the observability report? (*.report.json` /
`*.report.md` / `*.summary.md`)
docs\INGESTION_PIPELINE.md:156:per-video **observability report** (next to the video for
`import-local`; into the
docs\annotation\ANNOTATION_GUIDE.md:123:This is a **separate, observational** column (no
judgment): just look at the last shot.
src\reporting\__init__.py:1:"""Per-video observability reports for the collector.
src\reporting\__init__.py:3:`obsreport` is a SOFT dependency: absent -> these emitters no-op and
the CLI runs
src\reporting\__init__.py:13:   import obsreport # noqa: F401
src\reporting\__init__.py:16:   OBSREPORT_AVAILABLE = True
src\reporting\__init__.py:17:except Exception: # pragma: no cover - only when obsreport is
absent
src\reporting\__init__.py:18:   OBSREPORT_AVAILABLE = False
src\reporting\__init__.py:33:   """Write the delivery observability report into out_dir
(alongside the existing
src\reporting\__init__.py:35:   if not OBSREPORT_AVAILABLE:
src\reporting\__init__.py:59:   """Write the import observability report next to the imported
video. If the video
src\reporting\__init__.py:62:   if not OBSREPORT_AVAILABLE:

```

```

src\reporting\spec.yaml:1:# Observability report spec for the sports-data-collector.
src\reporting\spec.yaml:2:# The report ENGINE lives in the shared `obsreport` library; this is
the
tests\test_reporting.py:1:"""Tests for the collector observability reports. obsreport is a soft
dependency;
tests\test_reporting.py:8:pytest.importorskip("obsreport")
tests\test_reporting.py:51:     from obsreport import render_customer, render_technical
tests\test_proxy.py:34:     # Scrub-friendly + size knobs at their ratified defaults.
tests\test_proxy.py:44:def test_build_command_custom_knobs():
tests\test_reporting_faq.py:9:pytest.importorskip("obsreport")
tests\test_reporting_faq.py:10:from obsreport import load_spec # noqa: E402
src\reporting\adapter.py:1:"""Adapt collector data into an obsreport envelope.
src\reporting\adapter.py:5:report shape. Imported only after ``src.reporting`` confirms
obsreport is present,
src\reporting\adapter.py:6:so it may import obsreport freely.
src\reporting\adapter.py:13:import obsreport
src\reporting\adapter.py:23:def load_spec() -> "obsreport.ReportSpec":
src\reporting\adapter.py:24:     return obsreport.load_spec(_SPEC_PATH)
src\reporting\adapter.py:32:def issue_to_flag(issue: Any, record: Any = None, fps: Any = None)
-> "obsreport.Flag":
src\reporting\adapter.py:50:     return obsreport.Flag(
src\reporting\adapter.py:61:def _recorder(source: str, video_path: Optional[str], spec) ->
"obsreport.RunRecorder":
src\reporting\adapter.py:62:     return obsreport.RunRecorder(
src\reporting\adapter.py:82:) -> "obsreport.RunRecorder":

```

[Showing results with pagination = limit: 40]

10. user (2026-06-12T10:45:27.673Z)

```

diff --git a/docs/INGESTION_PIPELINE.md b/docs/INGESTION_PIPELINE.md
index 3120872..a4f9cf5 100644
--- a/docs/INGESTION_PIPELINE.md
+++ b/docs/INGESTION_PIPELINE.md
@@ -22,9 +22,15 @@ to the annotation spec (`docs/annotation/ROORA_BRIEF.md` +
`ANNOTATION_GUIDE.md`
                                     slice clips / calibrate / train
...

-## The four commands (in order)
+## The five commands (in order)

```bash
+# 0. BEFORE handoff: re-encode the source into a frame-identical annotation proxy
+# (ADR-002 ? downscaled + scrub-friendly so the vendor can work at frame step 1;
+# same frame count/fps/timeline, parity-verified, provenance recorded).
+# Share the PROXY with the vendor; everything downstream keeps using the ORIGINAL.
+python -m src.cli.curate prep-annotation-proxy --video-path match.mp4
+
1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a CSV)
python -m src.cli.curate convert-cvat --input job.xml \
 --video-path match.mp4 --out canonical.csv --issues issues.txt
@@ -118,8 +124,25 @@ python -m backend.tools.rally_slicer --video match.mp4 \
Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).

Does coarse frame sampling hurt boundaries?
-Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
-padding, so we don't flag sampling density.
+It depends on the consumer. For highlight compilation, ~0.5s sampling slack is
+absorbed by the compiler's clip padding, so validate-delivery doesn't flag sampling
+density. For golden training/eval labels the bar is $\pm 0.3s$ (Brief §4), and the
+pickleball/table-tennis pilot showed a 30-frame sampling grid at 59.94 fps puts every
+boundary ?0.5s off ? structurally out of spec. That's why videos are handed off as
+annotation proxies (step 0): light enough to annotate at frame step 1, with the
+first/last box of each rally on the exact contact/dead frame.

```

```

+
+### What's an annotation proxy, and why not annotate the original?
+A 4K/59.94 original is too heavy to frame-step in CVAT ? at Phase-B scale (~30-minute
+videos, >100k frames each) it's not realistic at all, which is what pushed the pilot
+vendor onto the coarse sampling grid above. `prep-annotation-proxy` re-encodes the
+source downsampled (?1080p) with dense keyframes for snappy scrubbing while preserving
+the **exact frame count, fps, and timeline** (verified after the encode; a failed proxy
+is deleted). Frame N on the proxy IS frame N on the original, so returned labels apply
+to the original unchanged. Provenance (source + proxy MD5) is appended to
+`data/annotation_proxy_manifest.csv` ? commit it. The proxy is ONLY for annotation:
+`import-local`, `export`, and the indexer (which keys videos by file MD5) keep using
+the original. Full rationale: ADR-002 in `docs/DECISIONS.md`.

Will a non-commercial license block export?
The video is **kept** in the DB regardless of license, but the default `export` is

```

#### 11. user (2026-06-12T10:45:41.548Z)

```

C:\Users\avidu\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Microsoft.Winget.Source_8weky
b3d8bbwe\ffmpeg-8.1.1-full_build\bin\ffmpeg.exe
C:\Users\avidu\AppData\Local\Microsoft\WinGet\Packages\Gyan.FFmpeg_Microsoft.Winget.Source_8weky
b3d8bbwe\ffmpeg-8.1.1-full_build\bin\ffprobe.exe
ffmpeg version 8.1.1-full_build-www.gyan.dev Copyright (c) 2000-2026 the FFmpeg developers

```

#### 12. user (2026-06-12T10:45:51.924Z)

```

{
 "programs": [
],
 "stream_groups": [
],
 "streams": [
 {
 "r_frame_rate": "30/1",
 "avg_frame_rate": "30/1",
 "nb_read_packets": "60"
 }
],
 "format": {
 "duration": "3.023000"
 }
 }
 }
}

```

#### 13. user (2026-06-12T10:46:06.856Z)

```

frame= 60 fps=0.0 q=-1.0 Lsize= 147KiB time=00:00:01.93 bitrate= 623.1kbits/s speed=12.4x
elapsed=0:00:00.15
using C:\Users\avidu\AppData\Local\Temp\proxyrep/src_tail.mkv True
SOURCE ProbeInfo: ProbeInfo(width=320, height=240, avg_fps=30.0, r_fps=30.0, nb_frames=60,
duration=3.023)
PROXY ProbeInfo: ProbeInfo(width=320, height=240, avg_fps=30.0, r_fps=30.0, nb_frames=60,
duration=2.0)
verify_parity errors: ['duration changed: source 3.023s -> proxy 2.000s (tolerance 0.200s)']

```

#### 14. assistant (2026-06-12T10:46:45.056Z)

Reproduced exactly with the branch's own code and the repo's live ffmpeg 8.1.1. Let me clean up and finalize.

#### 15. user (2026-06-12T10:46:45.592Z)

(Bash completed with no output)

**16. user (2026-06-12T10:47:17.981Z)**

Structured output provided successfully

**17. assistant (2026-06-12T10:47:25.085Z)**

Verification complete. The claim is real: reproduced end-to-end with the branch's own  
`probe\_video`/`build\_proxy\_command`/`verify\_parity` against ffmpeg 8.1.1 ? an MKV source with a  
1s audio tail probes as duration 3.023s (container fallback, video stream has no duration  
field), the frame-identical MP4 proxy probes as 2.000s (stream duration), and `verify\_parity`  
falsely reports "duration changed" (tolerance 0.2s), which `prep\_annotation\_proxy`  
(src/cli/curate.py:708-720) answers by deleting the proxy and hard-exiting with no override  
flag. Severity medium: fail-closed (no label poisoning possible) and current repo inputs are  
MP4, but the tool is the mandatory step-0 for every vendor handoff and accepts arbitrary  
sources.